

Функциональное программирование

Haskell

Охранные выражения (guards)

- это способ выбрать ветку вычисления по булевому условию.

Записываются через `|` после имени функции и аргументов.

GHC проверяет условия сверху вниз и берёт первое истинное.

otherwise — синоним **True**, ставится последним как «ветка по умолчанию».

Если ни одно условие не сработало, а **otherwise** нет — программа падает во время выполнения.

```
classify :: Int -> String
classify x
  | x > 0      = "положительное"
  | x == 0     = "ноль"
  | otherwise = "отрицательное"
```

case-выражение

case делает то же сопоставление с образцом, но внутри выражения, а не на уровне определения функции.

Удобен, когда нужно выбрать значение в середине вычисления или когда аргумент для сопоставления получается из другого выражения.

После **->** пишется результат для данного образца;

_ — любое значение, что в данном случае соответствует дефолтному кейсу.

```
applyOp :: Int -> Int -> Int -> Int
applyOp code x y = case code of
  1 -> x + y
  2 -> x - y
  3 -> x * y
  _ -> 0
```

if-then-else

if-then-else — это выражение, а не оператор: оно всегда возвращает значение.

Условие — после **if**, результат при истинности — после **then**, при ложности — после **else**.

Обе ветки обязательны и должны иметь одинаковый тип.

Пропустить **else** нельзя: компилятор не поймёт, какой тип возвращать.

```
safeDiv :: Int -> Int -> Double
safeDiv x y =
  if y == 0
    then 0.0
    else fromIntegral x / fromIntegral y
```

```
toPercent :: Int -> Int -> Percent
toPercent part total =
  if total == 0 then 0.0 else fromIntegral part /
  fromIntegral total * 100
```

let и where

let и **where** — два способа задать локальные привязки (аналог локальных переменных).

where пишется после тела функции и виден во всём определении, включая охранные условия.

let ... in пишется внутри выражения: то, что до **in**, видно только в выражении после **in**.

Оба — ленивые: значение вычисляется, только когда оно впервые потребуется.

```
squarePlus5 :: Int -> Int
squarePlus5 x = x*x + offset
  where
    offset = 5
```

```
squarePlus5' :: Int -> Int
squarePlus5' x =
  let offset = 5
  in x*x + offset
```

```
calc :: Int -> Int
calc x =
  let a = x + 1
      b = a * 2
  in b - 3
```

Лямбда-выражения

Лямбда — анонимная функция без имени.

Записывается как `\аргументы -> тело`.

Многоаргументные лямбды: `\x y -> ....`

Чаще всего используются в `map`, `filter`, `fold`
— там, где функция нужна один раз и не заслуживает отдельного имени.

```
addOne :: Int -> Int
addOne = \x -> x + 1
```

```
incList :: [Int] -> [Int]
incList = map (\x -> x + 1)
```

```
filterPos :: [Int] -> [Int]
filterPos = filter (\x -> x > 0)
```

СИМВОЛ @

@ связывает имя с полным значением, которое сопоставляется с образцом.

Пишется так: **имя@образец**.

Полезно, когда нужно и проверить структуру (через образец), и иметь доступ ко всему значению целиком без дублирования вычислений или без повторного конструирования.

Работает в любых местах, где разрешён pattern matching (сопоставление с образцом): в определениях функций, в **case**, в **let-привязках**, в аргументах лямбд и т. д.

```
data Box = Box Int Int
```

```
areaAndBox :: Box -> (Int, Box)
```

```
areaAndBox b@(Box w h) = (w * h, b)
```

```
b@(Box w h) :
```

b — вся коробка;

w, **h** — ширина и высота для площади.

Для `areaAndBox (Box 3 5)` получим `(15, Box 3 5)`.

Частые ошибки и нюансы

- Нельзя писать @ без образца: `x@` — ошибка.
Правильно: `x@образец`.
- Нельзя ставить @ внутри вложенных скобок произвольно: синтаксис строго `имя@шаблон`.
- Это не оператор, а часть синтаксиса паттернов: @ не используется отдельно в выражениях, только в сопоставлении с образцом.